ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

**Mestrado em Engenharia Informática – Cibersegurança e Administração de Sistemas.**

# LABORATÓRIO DE CIBERSEGURANÇA
# EMBEDDED LAB: INSECURE VENDING MACHINE

LABCIB

**1220638 Diogo Gomes**
**1250515 Gabriel Costa**
**1131504 Joana Soares**

26/10/2026

Instituto Superior de Engenharia do Porto

# Resumo

Este relatório documenta a avaliação de segurança de um sistema embebido, uma máquina de vending ligada à internet. O trabalho está dividido em sete targets: recuperação da password de developer por timing attack, dump e análise do firmware (strings, entropy e material criptográfico), recuperação da password do C&C e exploração do gerador de OTP, e por fim a ligação ao servidor de comando e controlo e a obtenção do secret final. Todas as evidências são produzidas na placa Arduíno atribuída ao grupo.

**Palavras-chave:** timing attack; canais laterais; firmware; entropy; RSA; MD5; rainbow table; OTP; C&C

# Índice

*Índice de Figuras* ..... 5

*Índice de Tabelas* ..... *Erro! Marcador não definido.*

*Glossário* ..... 7

*1 Introdução* ..... 8

*2 Objetivos e Âmbito* ..... 9

*3 Ambiente e Ferramentas* ..... 10

*4 Metodologia* ..... 11

*5 Procedimentos* ..... 12

5.1 Target 1 - Password de Developer ..... 12

5.2 Target 2 - Dump do Firmware ..... 16

5.3 Target 3 - Strings confidenciais ..... 18

5.4 Target 4 - Dados criptográficos ..... 20

5.5 Target 5 - Password do C&C ..... 22

5.6 Target 6 - Gerador OTP ..... 23

5.7 Target 7 - Secret Final ..... 25

*6 Resultados e Evidências* ..... 27

6.1 Target 1 - Password de Developer ..... 27

6.2 Target 2 - Firmware Dumped ..... 28

6.3 Target 3 - Strings Confidenciais ..... 28

6.4 Target 4 - Dados Criptográficos ..... 29

6.5 Target 5 - Password do C&C ..... 30

6.6 Target 6 - Gerador OTP ..... 30

6.7 Target 7 - Secret Final ..... 31

*7 Conclusões* ..... 32

*8 Referências* ..... 33

# Índice de Figuras

*Figura 1 - Consola da placa 22 no Serial Monitor (prompt e "Login failure").* ..... 12

*Figura 2 - Teste de ligação à consola da placa 22 (resposta "Login failure").* ..... 13

*Figura 3 - timing_attack.py scan devolve o número de caracteres da password, neste caso 13 e a 1ª letra 'd'.* ..... 14

*Figura 4 - Varredura comprimento x 1.º carácter: o comprimento 13 destaca-se. (Máquina 22)* ..... 14

*Figura 5 - Recuperação carácter a carácter (margem ~1 ms por posição).* ..... 15

*Figura 6 - Timing por carácter na 1.ª posição (mínimo): 'd' destaca-se. (Máquina 22)* ..... 15

*Figura 7 - Timing por carácter na 2.ª posição (mínimo): 'a' destaca-se. (Máquina 22)* ..... 15

*Figura 8 - A password completa "dansudeulzblj" abre a consola de developer.* ..... 16

*Figura 9 - Menu de developer: listagem das regiões e seleção para dump.* ..... 16

*Figura 10 - Captura do PROGMEM: 32768 bytes e SHA-256.* ..... 17

*Figura 11 - Verificação independente do dump: SHA-256 (certutil), tamanho, offset e vetores AVR.* ..... 17

*Figura 12 - Reprodutibilidade: um segundo dump dá o mesmo SHA-256.* ..... 17

*Figura 13 - Tabela de credenciais no strings.txt: Developer Dan (offset 18224) e Admin Alan (offset 18249); o offset relativo da password do Alan é 25 bytes.* ..... 19

*Figura 14 - Diálogo Dan/Alan no strings.txt com a chave privada (prefixo MII, 1592 caracteres) e a assinatura (344 caracteres = 256 bytes).* ..... 19

*Figura 15 - Saída do openssl asn1parse: SEQUENCE, version, modulus de 257 bytes (2048 bits) e expoente 010001 (65537).* ..... 20

*Figura 16 - Entropy do Firmware (Máquina 22)* ..... 21

*Figura 17 - Entropy do Firmware (Máquina 38)* ..... 21

*Figura 18 - Decriptação da assinatura com a chave privada: o bloco contém o hash MD5 "e07fcd7e6951d04659c4a8debe8e63de"* ..... 22

*Figura 19 - Procura do MD5 na rainbow table: a password do C&C corresponde ao hash recuperado.* ..... 22

*Figura 20 - Exploração manual no Serial Monitor: login como Admin Alan leva ao fluxo do C&C e, com OTPs erradas, a placa revela a OTP esperada em cada tentativa (a fuga explorada).* .... 23

*Figura 21 - Execução do otp_crack.py: 1024 OTPs capturadas (período 1024), gerador recuperado a=3693 c=0 m=4096 e próxima OTP prevista*..... 24

*Figura 22 – Máquina 22: Ligação ao C&C bem-sucedida, Secret Final "calmvacationquiltmonkey"* ..... 25

*Figura 23 - Máquina 38: Ligação ao C&C bem-sucedida, Secret Final "mittenamazingmonkeyzipper"*..... 25

# Glossário

| | |
|---|---|
| **Timing attack** | Ataque por canal lateral que infere informação secreta a partir das diferenças no tempo de resposta do sistema. |
| **Canal lateral** | Fuga de informação através de características de execução (tempo, consumo, etc.) e não da interface lógica. |
| **Firmware** | Software gravado no dispositivo que controla o seu funcionamento. |
| **PROGMEM** | Região de memória de programa (Flash) dos microcontroladores AVR onde ficam código e dados constantes. |
| **Entropy** | Medida de aleatoriedade de uma região de dados; valores altos indicam dados cifrados ou comprimidos. |
| **Base64** | Codificação que representa dados binários em texto ASCII. |
| **ASN.1 / DER** | Notação e codificação binária usadas para estruturar chaves e assinaturas criptográficas. |
| **RSA** | Algoritmo de criptografia assimétrica usado para cifra e assinatura digital. |
| **MD5** | Função de hash de 128 bits, hoje insegura devido a colisões. |
| **Rainbow table** | Tabela pré-computada de hashes usada para inverter hashes e recuperar os valores originais. |
| **OTP** | One-Time Password: código de uso único. |
| **PRNG** | Gerador de números pseudoaleatórios; previsível se o estado ou os parâmetros forem conhecidos. |
| **C&C** | Servidor de comando e controlo com que o dispositivo comprometido comunica. |
| **SHA-256** | Função de hash criptográfico de 256 bits usada para verificar integridade. |

# 1 Introdução

A empresa adquiriu uma máquina de *vending* para o seu edifício de alta tecnologia. Por exigência contratual, a máquina mantém uma **ligação permanente à internet**, usada para a reposição automática de stock e para garantir a coerência entre os produtos comprados e os disponíveis.

Como não é aceitável ter um equipamento potencialmente comprometido com acesso à internet junto de projetos sensíveis, a equipa de cibersegurança iniciou a investigação do dispositivo. Partindo do trabalho preliminar, que apontou possíveis vulnerabilidades, este trabalho explora-as de forma controlada e documentada.

As tarefas foram realizadas em dois Arduinos, **Vending Machine 22** e **Vending Machine 38**. Por uma questão de exemplo e para evitar repetições ao longo do relatório, a análise foca-se principalmente na placa nº22, apresentando-se também os resultados da nº38 sempre que relevante. O trabalho está organizado em sete targets, desde a **recuperação de credenciais** até à ligação final ao **servidor de comando e controlo (C&C)**.

# 2 Objetivos e Âmbito

O objetivo deste trabalho é avaliar e explorar a segurança de um sistema embebido, demonstrando o impacto prático de cada vulnerabilidade identificada. O âmbito cobre os sete targets do enunciado corridos em 2 Arduínos diferentes:

- **Target 1:** recuperar a password de *developer* (Dan) através de um *timing attack* à consola série.
- **Target 2:** obter o *dump* do *firmware* a partir da consola de desenvolvimento.
- **Target 3:** extrair e analisar as strings do *firmware*, incluindo a segunda password de manutenção (Alan).
- **Target 4:** identificar o material criptográfico presente no *firmware* (chave e assinatura).
- **Target 5:** recuperar a password do C&C a partir da assinatura e de uma *rainbow table*.
- **Target 6:** explorar o gerador de OTP e prever o próximo código.
- **Target 7:** ligar ao servidor de C&C e obter o *secret final*.

Todo o trabalho é realizado nas placas Arduíno atribuídas ao grupo (ID 22 e ID 38), num ambiente de laboratório autorizado.

# 3 Ambiente e Ferramentas

O trabalho foi realizado no seguinte ambiente:

- **Hardware:** Arduíno Nano [1], placas com ID 22 e ID 38 ligadas por USB na porta COM3 e COM5, respetivamente.
- **Ligação:** USB e consola série (Serial Monitor ou terminal), a 115200 baud (9600 em alternativa).
- **Comunicação e automação:** Python 3 com as bibliotecas pyserial (medição de tempos e sessões série) e matplotlib (gráficos do *timing attack* e da *entropy*). [2]
- **Análise de firmware:** strings (Sysinternals, no Windows) para o inventário de strings, certutil (nativo do Windows; sha256sum/shasum em Linux/macOS) para o SHA-256, e um script próprio de cálculo de *entropy*.
- **Criptografia:** OpenSSL (openssl asn1parse, openssl rsa -text e operações de decriptação).
- **Rainbow table:** a tabela curta fornecida no contexto da lab.

As versões concretas das ferramentas são indicadas no README do código.

# 4 Metodologia

A metodologia seguiu a ordem dos targets, uma vez que cada um depende do anterior, em ambos os arduínos.

- **Target 1 (timing attack):** a validação da password demora mais tempo quanto mais caracteres iniciais estiverem corretos, porque a comparação termina no primeiro carácter errado. Começa-se por inferir o comprimento, comparado antes do resto, e depois, posição a posição, escolhe-se o carácter com maior tempo médio de resposta, repetindo várias amostras para reduzir o ruído. Assume-se [A-Z] e no máximo 20 caracteres. [3]
- **Target 2 (dump):** com a consola de developer acessível, usa-se a sua funcionalidade para ler a memória de programa (PROGMEM) e extrair o firmware. Cada binário é guardado sem alterações e calcula-se o SHA-256 para garantir integridade e permitir verificação.
- **Target 3 (strings):** sobre o dump corre-se strings para revelar texto em claro, procurando passwords, nomes, URLs e blobs em base64. Cada asset relevante é anotado com a sua função.
- **Target 4 (cripto):** identifica-se o material criptográfico pela forma como está codificado (por exemplo base64) e pela entropy, que distingue zonas de chaves ou dados cifrados das zonas de código. A chave é inspecionada com openssl asn1parse e openssl rsa -text, usando os campos estruturais ASN.1/DER para determinar o tipo e o tamanho.
- **Target 5 (password do C&C):** a partir da assinatura e da chave obtém-se, com openssl, um hash (MD5). Como o espaço de passwords é reduzido, uma rainbow table curta é suficiente para inverter o hash e recuperar a password do C&C.
- **Target 6 (OTP):** o gerador produz códigos diferentes a cada arranque, mas um bug leva à repetição após algumas tentativas. Capturando a sequência numa sessão série, recuperam-se os parâmetros do gerador e prevê-se o próximo valor; a captura e os parâmetros têm de ser do mesmo arranque.
- **Target 7 (secret final):** com a password do C&C (Target 5) e um OTP válido do arranque atual (Target 6), estabelece-se a ligação ao servidor de comando e controlo, registando a sessão até à linha do secret final.

# 5 Procedimentos

Os procedimentos detalhados de cada target, com comandos, configuração e evidências, são documentados nesta secção à medida que o trabalho é executado na placa.

## 5.1 Target 1 - Password de Developer

**Ligação à consola:**

- Arduíno Nano (Nº22) na porta COM3, a 115200 baud, terminação de linha New Line.
- Arduíno Nano (Nº38) na porta COM5, a 115200 baud, terminação de linha New Line.

**Prompt (após reset):** "Enter password for vending machine number 22:" (ou *machine* number 38)

**Resposta a password errada:** "Login failure", seguida de novo *prompt*.

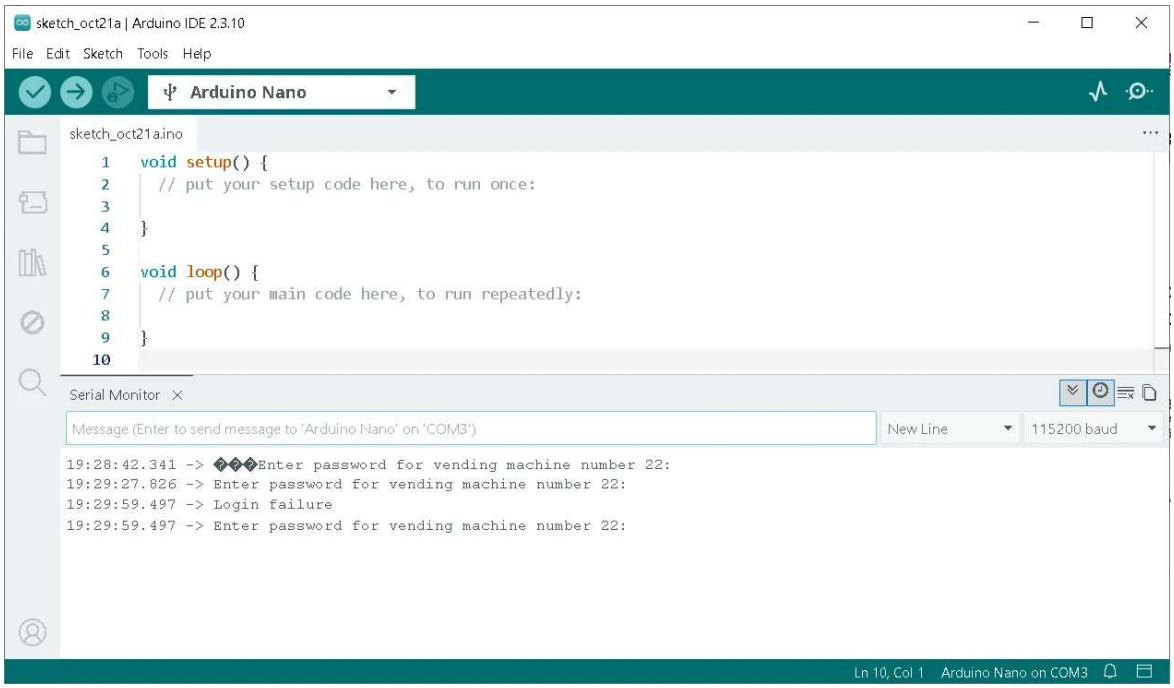



***Figura 1 - Consola da placa 22 no Serial Monitor (prompt e "Login failure").***

**Automação:** medição de tempos e comunicação série pelos scripts em code/target-1/ (serial_target.py, timing_attack.py, plot_timing.py).

```
cd code/target-1
pip install pyserial matplotlib   #instalar biblioteca
python timing_attack.py scan   # comprimento + 1o caracter
python timing_attack.py recover 13   # password completa
python timing_attack.py finish dansudeulzbl   # ultimo caracter
python plot_timing.py chars   # graficos 1a/2a letra
python plot_timing.py scan   # grafico da varredura
```

**Procedimento:** inferir comprimento e 1.ª letra (timing_attack.py scan), recuperar carácter a carácter (timing_attack.py recover 13) e confirmar o último pela resposta (timing_attack.py finish); gráficos com plot_timing.py.

```
PS C:\Users\dgrua\Desktop\MEI\2º ano\LABCIB\LABCIB_1131504_1250515_1220638\code\target1> python serial_target.py
elapsed = 62.393 ms
resposta: b'Login failure\r\nEnter password for vending machine number 22: \r\n'
```

***Figura 2 - Teste de ligação à consola da placa 22 (resposta "Login failure").***

**Medição:** cada tentativa é cronometrada do envio até à resposta; usam-se várias amostras por candidato e o mínimo, mais robusto ao ruído do sistema operativo do que a média.

**Inferência do comprimento:** enviar só filler (aaaa...) é inconclusivo: se o 1.º carácter não for 'a', todas as tentativas falham no mesmo ponto e dão tempos iguais. Usa-se por isso uma varredura combinada: para cada comprimento possível testa-se a-z na 1.ª posição e, no comprimento certo, um carácter separa-se dos restantes (o atraso do carácter correto).

**Nota de ambiente:** a latência da porta série no Windows pode mascarar diferenças pequenas; o chip CH340 do Nano não expõe *Latency Timer*, pelo que a robustez vem do número de amostras e do uso do mínimo. [4]

**Resultado da varredura:** o comprimento 13 destaca-se com uma separação de ~2 ms, enquanto os restantes ficam em ruído (<0,1 ms). A 1.ª letra provável é 'd'. Isto confirma a existência de um atraso por carácter correto, que é o que torna o ataque viável.

```
PS C:\Users\dgrua\Desktop\MEI\2º ano\LABCIB\LABCIB_1131504_1250515_1220638\code\target1> python timing_attack.py scan
len  1: melhor 'x'  gap p/ 2o =  0.026 ms
len  2: melhor 'k'  gap p/ 2o =  0.004 ms
len  3: melhor 'd'  gap p/ 2o =  0.008 ms
len  4: melhor 'f'  gap p/ 2o =  0.006 ms
len  5: melhor 'z'  gap p/ 2o =  0.033 ms
len  6: melhor 'u'  gap p/ 2o =  0.013 ms
len  7: melhor 'b'  gap p/ 2o =  0.007 ms
len  8: melhor 'm'  gap p/ 2o =  0.012 ms
len  9: melhor 'r'  gap p/ 2o =  0.013 ms
len 10: melhor 'r'  gap p/ 2o =  0.006 ms
len 11: melhor 'g'  gap p/ 2o =  0.024 ms
len 12: melhor 'e'  gap p/ 2o =  0.015 ms
len 13: melhor 'd'  gap p/ 2o =  1.979 ms
len 14: melhor 'm'  gap p/ 2o =  0.003 ms
len 15: melhor 't'  gap p/ 2o =  0.003 ms
len 16: melhor 'z'  gap p/ 2o =  0.013 ms
len 17: melhor 's'  gap p/ 2o =  0.001 ms
len 18: melhor 'x'  gap p/ 2o =  0.008 ms
len 19: melhor 'p'  gap p/ 2o =  0.039 ms
len 20: melhor 'g'  gap p/ 2o =  0.012 ms

Maior separacao: comprimento 13, 1o caracter provavel 'd' (gap 1.979 ms)
Se esse gap for claramente maior que os restantes, e esse o comprimento.
Dados brutos -> scan_pos0.csv
```

***Figura 3 - timing_attack.py scan devolve o número de caracteres da password, neste caso 13 e a 1ª letra 'd'.***

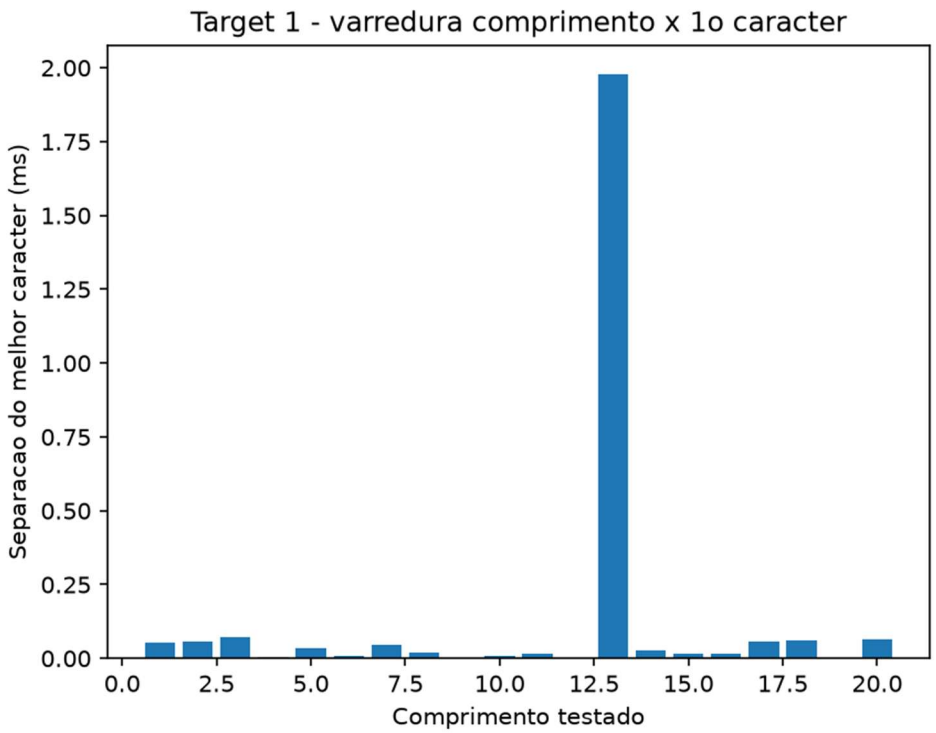



***Figura 4 - Varredura comprimento x 1.º carácter: o comprimento 13 destaca-se. (Máquina 22)***

**Recuperação carácter a carácter:** fixando o comprimento 13, cada posição foi recuperada escolhendo o carácter de maior tempo; a margem do vencedor é ~1 ms. A password obtida para a **Vending Machine 22** é “dansudeulzblj” e para a **Vending Machine 38** é “danzhbmzndzdd”.

```
PS C:\Users\dgrua\Desktop\MEI\2º ano\LABCIB\LABCIB_1131504_1250515_1220638\code\target1> python timing_attack.py recover 13
pos 0: 'd'  (margem p/ 2o lugar = 1.997 ms)  parcial = d
pos 1: 'a'  (margem p/ 2o lugar = 0.972 ms)  parcial = da
pos 2: 'n'  (margem p/ 2o lugar = 0.963 ms)  parcial = dan
pos 3: 's'  (margem p/ 2o lugar = 0.988 ms)  parcial = dans
pos 4: 'u'  (margem p/ 2o lugar = 0.978 ms)  parcial = dansu
pos 5: 'd'  (margem p/ 2o lugar = 0.975 ms)  parcial = dansud
pos 6: 'e'  (margem p/ 2o lugar = 0.948 ms)  parcial = dansude
pos 7: 'u'  (margem p/ 2o lugar = 0.958 ms)  parcial = dansudeu
pos 8: 'l'  (margem p/ 2o lugar = 0.969 ms)  parcial = dansudeul
pos 9: 'z'  (margem p/ 2o lugar = 0.959 ms)  parcial = dansudeulz
pos 10: 'b'  (margem p/ 2o lugar = 0.943 ms)  parcial = dansudeulzb
pos 11: 'l'  (margem p/ 2o lugar = 1.003 ms)  parcial = dansudeulzbl
```

***Figura 5 - Recuperação carácter a carácter (margem ~1 ms por posição).***

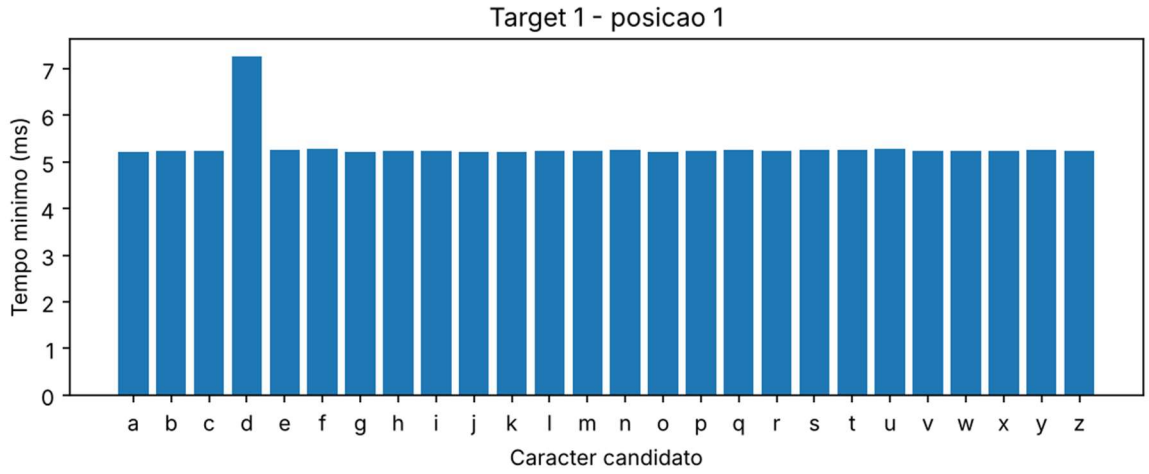



***Figura 6 - Timing por carácter na 1.ª posição (mínimo): 'd' destaca-se. (Máquina 22)***

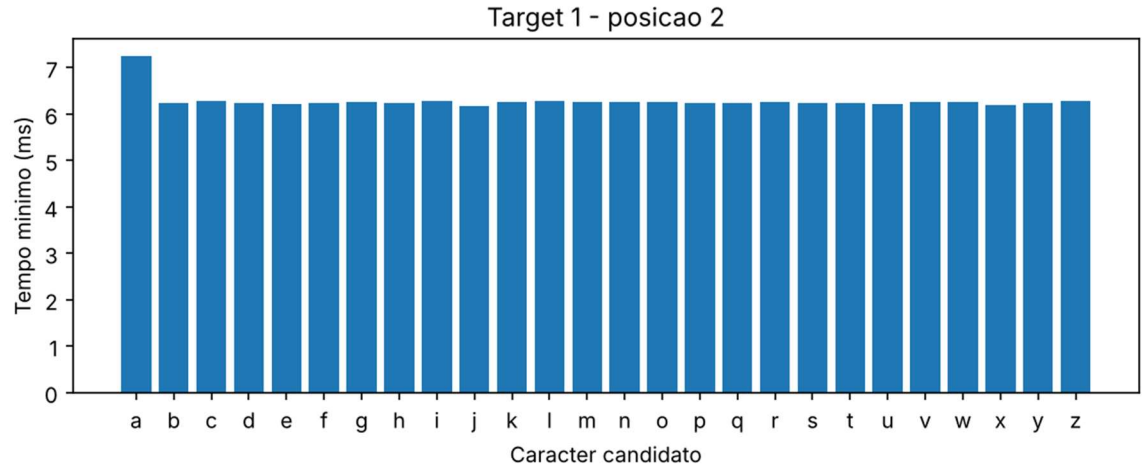



***Figura 7 - Timing por carácter na 2.ª posição (mínimo): 'a' destaca-se. (Máquina 22)***

**Último carácter:** confirmado pela resposta, não por timing: a password completa deixa de dar *'Login failure'* e abre a consola de *developer*.

```
PS C:\Users\dgrua\Desktop\MEI\2º ano\LABCIB\LABCIB_1131504_1250515_1220638\code\target1> python timing_attack.py finish dansudeulzbl
a: login failure  :: Enter password for vending machine number 22:  | Login failure | Enter password for vending machine number 22:  |
b: login failure  :: Login failure | Enter password for vending machine number 22:  |
c: login failure  :: Login failure | Enter password for vending machine number 22:  |
d: login failure  :: Login failure | Enter password for vending machine number 22:  |
e: login failure  :: Login failure | Enter password for vending machine number 22:  |
f: login failure  :: Login failure | Enter password for vending machine number 22:  |
g: login failure  :: Login failure | Enter password for vending machine number 22:  |
h: login failure  :: Login failure | Enter password for vending machine number 22:  |
i: login failure  :: Login failure | Enter password for vending machine number 22:  |
j: <<< DIFERENTE  :: ### END USER LICENSE AGREEMENT |  | #### **Congratulations, Thirsty Friend!** | By interacting with this vending machine

Password completa: dansudeulzblj
```

*Figura 8 - A password completa "dansudeulzblj" abre a consola de developer.*

## 5.2 Target 2 - Dump do Firmware

**Consola de developer:** após o login, o menu mostra [0] List Memory locations e [1] Dump Memory location.

**Regiões (List, opção 0):** PROGMEM (0 a 32767, 32768 bytes), EEPROM (1024 bytes) e SRAM (36864 bytes).

```
22:34:32.991 -> ============================
22:34:32.991 -> ==== DEVELOPMENT MENU ====
22:34:32.991 -> ============================
22:34:32.991 -> [0] List Memory locations
22:34:32.991 -> [1] Dump Memory location
22:37:33.184 ->     ==================================
22:37:33.217 ->     ==== Current memory locations ====
22:37:33.217 ->     ==================================
22:37:33.217 ->      (0) (0, 32767) PROGMEM: Flash memory for large data
22:37:33.217 ->      (1) (0, 1024) EEPROM: Memory for data storage
22:37:33.218 ->      (2) (0, 36863) SRAM: Memory for runtime
22:37:33.218 -> ============================
22:37:33.218 -> ==== DEVELOPMENT MENU ====
22:37:33.218 -> ============================
22:37:33.218 -> [0] List Memory locations
22:37:33.218 -> [1] Dump Memory location
22:38:04.150 ->     ==================================
22:38:04.150 ->     ==== MEMORY LOCATION TO PRINT ====
22:38:04.150 ->     ==================================
22:38:04.150 ->     [0] PROGMEM: Flash memory for large data
22:38:04.150 ->     [1] EEPROM: Memory for data storage
22:38:04.150 ->     [2] SRAM: Memory for runtime
```

*Figura 9 - Menu de developer: listagem das regiões e seleção para dump.*

**Dump (opção 1 e depois 0, PROGMEM):** os bytes saem em bruto (binário), não em hexadecimal; a consola vai imprimindo até se carregar Enter.

**Captura:** o script code/target-2/dump_capture.py faz login, navega o menu, lê exatamente 32768 bytes crus, grava progmem.bin e calcula o SHA-256.

```
cd code\target-2
python dump_capture.py 0
certutil -hashfile ..\..\TARGETS\target-2\progmem.bin SHA256
```

**Validação:** o binário começa por 0c 94 ... (tabela de vetores AVR), o que confirma firmware válido; o SHA-256 obtido na placa é igual ao recalculado sobre o ficheiro.




***Figura 10 - Captura do PROGMEM: 32768 bytes e SHA-256.***

**Verificação independente:** o SHA-256 recalculado com certutil sobre o progmem.bin coincide com o da captura; o ficheiro tem 32768 bytes, a password aparece no offset 700 e os primeiros bytes (0c 94 ...) são a tabela de vetores AVR.

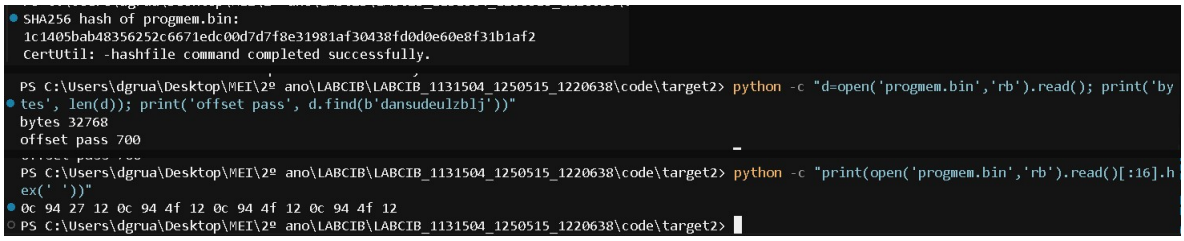



***Figura 11 - Verificação independente do dump: SHA-256 (certutil), tamanho, offset e vetores AVR.***

**Como se verifica o sucesso:**

1. O dump tem 32768 bytes, igual ao tamanho do PROGMEM;
2. Começa pelos vetores AVR (0c 94 ...), logo está alinhado desde o endereço 0;
3. O SHA-256 recalculado coincide;
4. Um segundo dump independente dá o mesmo SHA-256. Como a flash é estática, dois *dumps* iguais confirmam uma captura completa e sem bytes perdidos.

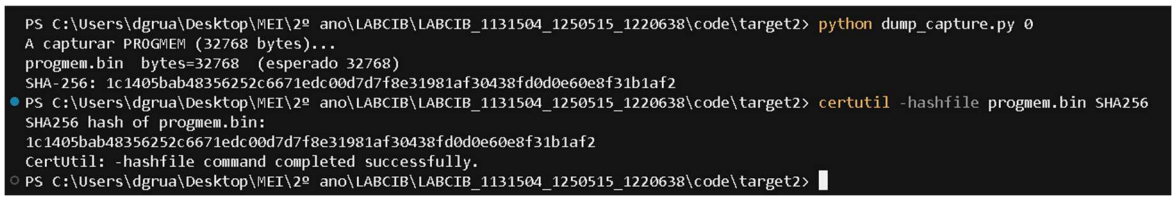



***Figura 12 - Reprodutibilidade: um segundo dump dá o mesmo SHA-256.***

**Resposta:** o dump do PROGMEM tem 32768 bytes e SHA-256: "1c1405bab48356252c6671edc00d7d7f8e31981af30438fd0d0e60e8f31b1af2" para a máquina 22 ou "89469672582076b77e07649c3427eadb8d6b1be12c2ef488bdaeca85822b6140" para a máquina 38.

A password de developer aparece em dois offsets: 700 (primeira ocorrência, no diálogo embebido) e 18224 (tabela de credenciais do login). No Target 3 usa-se o 18224 para o offset relativo (Alan em 18249 = Dan + 25).

## 5.3 Target 3 - Strings confidenciais

O Target 3 consiste em ler o que o firmware guarda em texto claro. O ponto de partida é o dump da flash obtido no Target 2 (progmem.bin). O passo-padrão é extrair as sequências de caracteres imprimíveis desse binário e inspecioná-las à procura de segredos. Para isso usou-se a ferramenta **strings** (Sysinternals, no Windows), que percorre o ficheiro e imprime cada sequência de texto legível. Para instalar o **strings** corremos o seguinte comando:

```
winget install Microsoft.Sysinternals.Suite
```

Para o próximo passo, a *flag* importante aqui é a **-o**, que faz o strings imprimir o ***offset*** (a posição em bytes) de cada string dentro do ficheiro. Sem o offset não seria possível responder à pergunta do offset relativo que o target pede, porque o Serial Monitor do Arduíno mostra o texto mas nunca a sua posição na memória.

```
cd TARGETS\target-2
strings64 -a -o -nobanner -n 4 progmem.bin > ..\target-
3\strings.txt
```

Cada flag: **-a** varre o ficheiro todo; **-o** imprime o offset de cada string; **-n 4** ignora sequências com menos de 4 caracteres, o que reduz o ruído; **-nobanner** remove o cabeçalho da ferramenta. O resultado fica em strings.txt, no formato offset:texto.

A inspeção faz-se sobre esse ficheiro no editor. Procurando por **Admin** chega-se à tabela de credenciais; procurando por **MII** chega-se à chave privada e, logo a seguir, à assinatura.

**Resultado:** a tabela mostra Developer Dan / “dansudeulzblj” no offset 18224 e Admin Alan / “alanimyirtjaawjwdkbzonruikuhxbmvsgykfvtkrirofdikmzvocg” no offset 18249. O offset relativo da password do Alan é portanto 18249 - 18224 = 25 bytes. A chave privada base64 tem 1592 caracteres e a assinatura 344, o que corresponde a 256 bytes, consistente com uma assinatura RSA-2048.

**Outras strings relevantes:** mensagens de login e do menu de dump, o fluxo de C&C (OTP, secret final) e o diálogo Dan/Alan que explica a cripto. Tudo fica registado em strings.txt e está anotado em TARGETS/target-3/README.md.

**Evidência:** as figuras seguintes mostram, no próprio strings.txt, a tabela de credenciais com os offsets (de onde sai o offset relativo +25) e os dois blobs base64 (chave e assinatura) gravados no firmware.

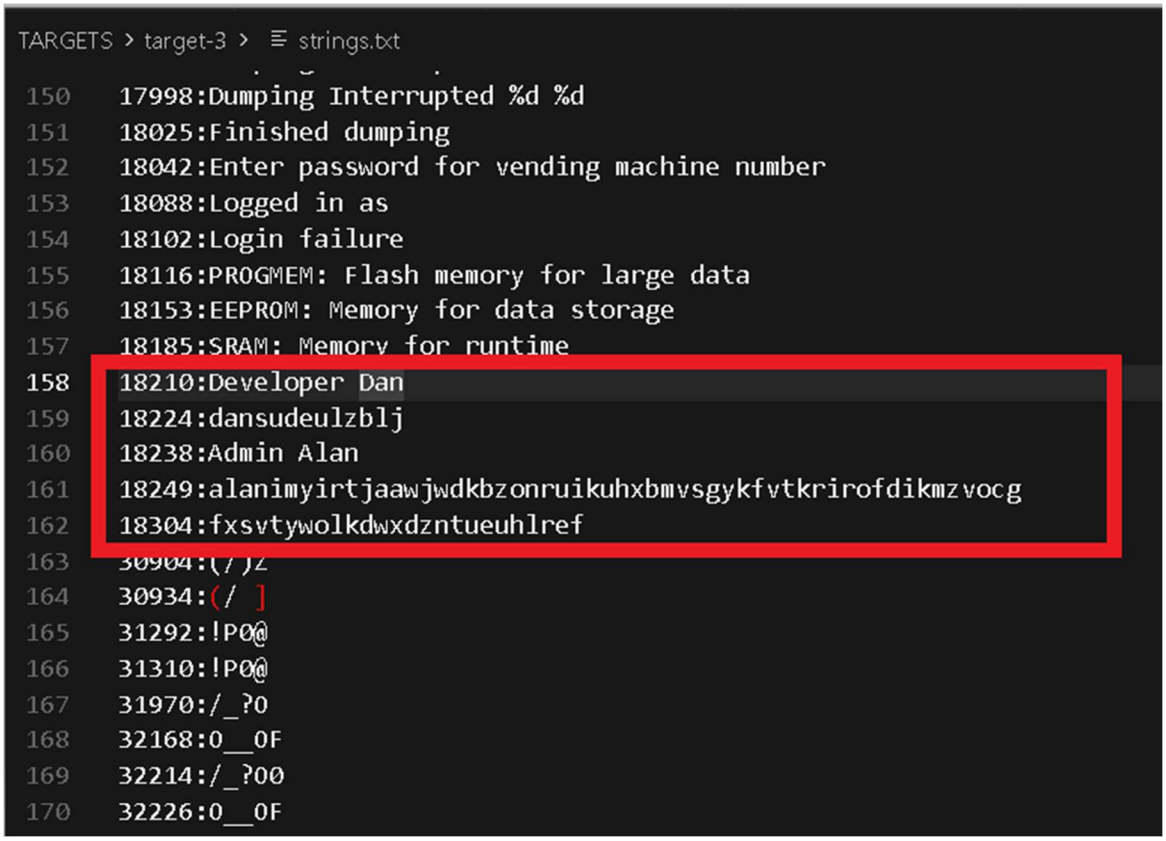



***Figura 13 - Tabela de credenciais no strings.txt: Developer Dan (offset 18224) e Admin Alan (offset 18249); o offset relativo da password do Alan é 25 bytes.***

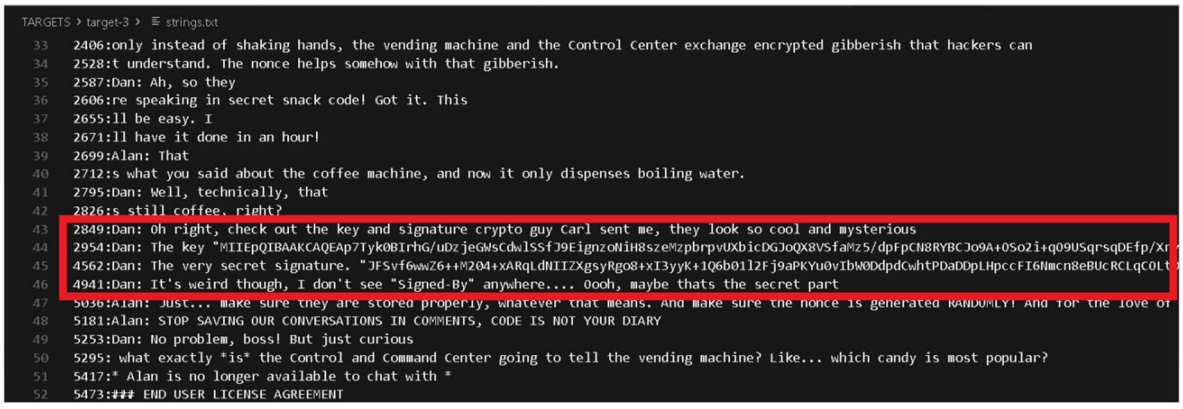



***Figura 14 - Diálogo Dan/Alan no strings.txt com a chave privada (prefixo MII, 1592 caracteres) e a assinatura (344 caracteres = 256 bytes).***

## 5.4 Target 4 - Dados criptográficos

**Localização:** na análise de strings do Target 3 destacam-se dois blobs base64, a chave (offset 2954) e a assinatura (offset 4562). Guardámos cada blob (o texto base64) em key.b64 e sig.b64.

**Descodificação e identificação:** descodificámos o base64 para binário (DER) e usámos o openssl para ler a estrutura interna da chave.

```
cd TARGETS\target-4
openssl base64 -d -A -in key.b64 -out key.der
openssl base64 -d -A -in sig.b64 -out sig.bin
openssl asn1parse -inform DER -in key.der
openssl rsa -inform DER -in key.der -text -noout
```

O **-A** é necessário porque o blob é uma linha única e longa (sem ele o openssl espera linhas de 64 caracteres). O **asn1parse** [5] percorre a estrutura DER campo a campo; o **rsa -text** interpreta-a como chave RSA.

**Resultado:** a chave é uma RSA-2048 privada. Os bytes que a identificam são: SEQUENCE (30 82...) que abre a RSAPrivateKey; INTEGER 00 (version); um INTEGER de 257 bytes que é o modulus (256 bytes + 1 byte de sinal, logo 2048 bits); e o INTEGER 010001 (expoente público 65537). O prefixo base64 MII corresponde aos bytes 30 82, o magic de uma estrutura DER. A assinatura (sig.bin) tem 256 bytes, exatamente um bloco RSA-2048.

```
PS C:\Users\dgrua\Desktop\MEI\2º ano\LABCIB\LABCIB_1131504_1250515_1220638\TARGETS\target-4> Get-Content asn1parse_key.txt -TotalCount 8
    0:d=0  hl=4 l=1189 cons: SEQUENCE
    4:d=1  hl=2 l=   1 prim: INTEGER           :00
    7:d=1  hl=4 l= 257 prim: INTEGER           :A7B4F2934048AE11BFB83CE37865AC09DC254927C9F448A09F3A0D887F2CCDE333A5BAE9BD45DB89C0C6268417F1549F
68CCF9FDDA45A4237C458042268F40F8E4A8DA2FAA3BD512AABB2A0C47E9FD7AF207A0DCCF660007D9369F0465D5F34844AE48F18B0C4DFBB4AF84D39FB9761DF13AB91BF15FBAD7
E588960914E1F7F9F05BA94F97E334EED165E12E4447A7E3789C8F4EB90909A0C51A1A2DCA7A96C37EF15F7AC0B918249AF88198D6460242934D636FDD11B8EF687AB10C37B2BD6C
B465696F45DC58298796503A6430F20FC83BA3F968FC8975C9033D2A6F1116B2CE861BC04597D9ACC05775C7381B91018770053153830D65EEB38D6A0BCF2163
  268:d=1  hl=2 l=   3 prim: INTEGER           :010001
  273:d=1  hl=4 l= 256 prim: INTEGER           :08D8FD81323F835B715B656EF4BF9C6B6ABF56D4300DE0B504A179EC4D154E896184A94F428A887204870FAA215F3C81
2A74C1ED9811DB7E79C46AE75284753F8C44B23A18878E8EAD0E31C2331B0FA1E859E9AD2C8DEB8B7E74A8C001255FEFDC9BBB426EFD07B2334E9D39D2EFE460E104A1966D4E091B
F1C29A73C133C8B2C390ED657F3BCE51A366655614C3346D2CDDFB736B21DF0F5648B7199A517C9C8F16726168F800D3971FAB4124FF33B55E4C43873D78E86FA40957DCCF5C42DA
B1FDA08F8C4E7BE7783E63BFD3D8ABB198497A01EE7986EE9B7FFB9850DDB1F9A8BFEBA7A577799693CE5DFA0FA15BB98316641A626C3311B18264CEECCCEE51
  533:d=1  hl=3 l= 129 prim: INTEGER           :D645C3F6DE699E4215BCB3CDFD75D3EB1424C9020C363BC112882E8010947859588E2ECCC29B8AD33B1ED8F1D5284F62
23CA0A3FFF1909CCFA4C916B9306912D62CD6C9C63D7EA858DB0F39AE822095F699C201544A9B170EDF09BC86773A944F3D3679DDD2933559B71A2FD18E23CD33A6D06B0E35F6A42
76515B8BDBBB2E93
  665:d=1  hl=3 l= 129 prim: INTEGER           :C85DB8264AD57881BC49A549E89DFC12A7D59966827333AA828CC885ACEF579BD13D228C5A0B803E1715B7409F6DD57C
71056A79B94F486F49B7383C3058EF0E0E27968F11FE7D444D92E12DD2B97629D63B896A02D29C2EE12CAFA6A1EAE317B0996CE6AF221F948860906115E68F88FAA5AFEC3DBF591A
C3B6FC7F420B33F1
  797:d=1  hl=3 l= 129 prim: INTEGER           :9510DA00EFC0389A2EF7257BA3DE09FD1BF96F619E4F4888CC34B36CE29BF791E8BA99B2579CD01A9DA19B39BA9A1D87
DFF019EB65B3EC8E8E7F2017DFA40CF1AF2FCC648C4D54D96EB670F30E954741FA7DE41007C1AA312F47598934F75106283EA5723CBF01F88778A9F43C1080524EEE345E71001B33
7B8EA40A8B661043
PS C:\Users\dgrua\Desktop\MEI\2º ano\LABCIB\LABCIB_1131504_1250515_1220638\TARGETS\target-4>
```

***Figura 15 - Saída do openssl asn1parse: SEQUENCE, version, modulus de 257 bytes (2048 bits) e expoente 010001 (65537).***

**Entropy:** para confirmar que ali está material criptográfico e não código, calculámos a entropy de Shannon em janelas de 256 bytes ao longo do firmware.

```
python code\target-4\entropy_scan.py
```

O código ronda 4,63 bits/byte; a chave em base64 ~5,98 e a assinatura ~5,89 (o teto do base64 é 6, porque usa 64 símbolos); a chave descodificada ~7,81 (binário quase aleatório). O material criptográfico destaca-se claramente do código.

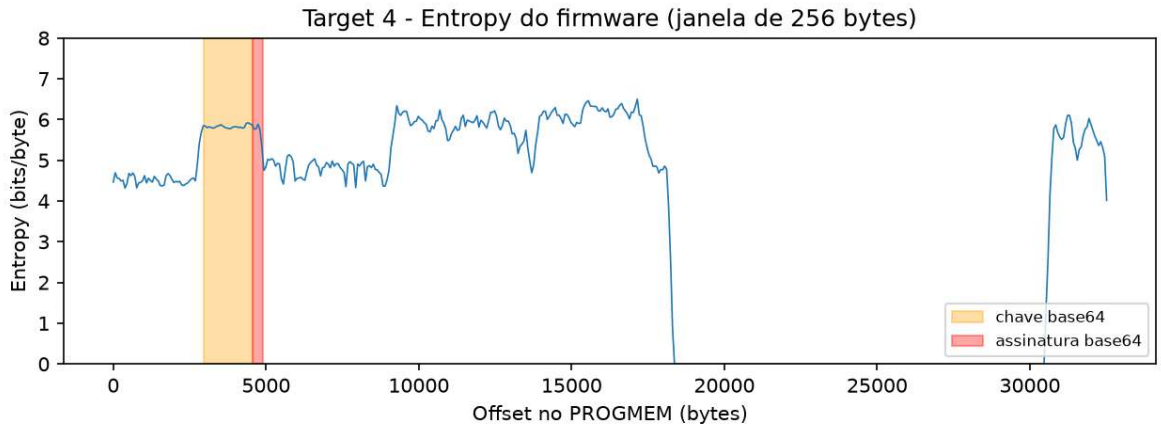



***Figura 16 - Entropy do Firmware (Máquina 22)***

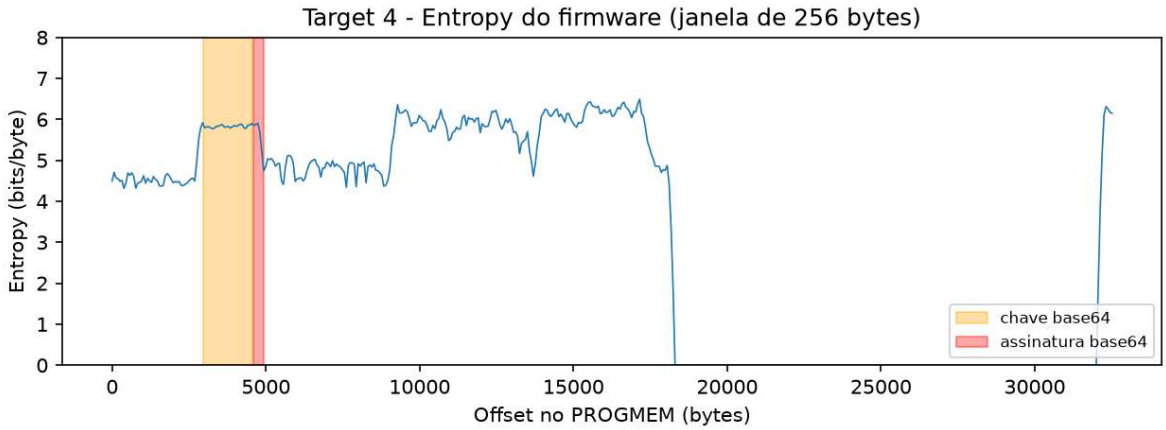



***Figura 17 - Entropy do Firmware (Máquina 38)***

## 5.5 Target 5 - Password do C&C

**Observação do Target 4:** a assinatura (sig.bin) não se comporta como assinatura. Aplicar a chave pública devolve um bloco aparentemente aleatório e o *padding* PKCS#1 falha; aplicar a chave privada devolve texto limpo. Conclui-se que é um bloco encriptado com a chave pública, e nós temos a chave privada (do Target 4) para o abrir.

**Decriptação:** decriptámos sig.bin com a chave privada.

```
cd TARGETS\target-5
openssl pkeyutl -decrypt -inkey ..\target-4\key.der -keyform DER -
in ..\target-4\sig.bin -out hash.json
type hash.json
```

O resultado é um JSON com um hash MD5: {"hash": "e07fcd7e6951d04659c4a8debe8e63de"}.

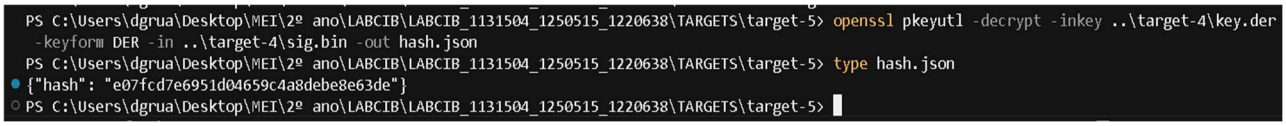



***Figura 18 - Decriptação da assinatura com a chave privada: o bloco contém o hash MD5 "e07fcd7e6951d04659c4a8debe8e63de"***

**Lookup na rainbow table:** procurámos esse MD5 na coluna de hashes da tabela fornecida (Rainbowtablee.txt).

```
findstr "e07fcd7e6951d04659c4a8debe8e63de"
..\..\..\Rainbowtablee.txt > rainbow_lookup.txt
```

A linha correspondente dá a password do C&C: "cncazezuxmihjpgrddzjnemmyzajtwcyqufqqrqcrsoqhsdtnnwwa". Confirmámos o round-trip com certutil: md5(password) = e07fcd7e6951d04659c4a8debe8e63de, o mesmo hash que saiu da assinatura.

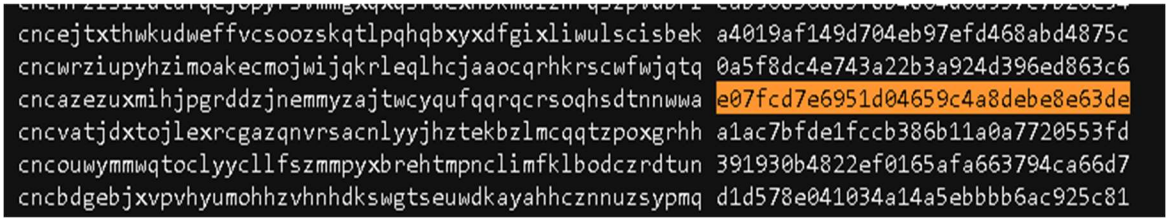



***Figura 19 - Procura do MD5 na rainbow table: a password do C&C corresponde ao hash recuperado.***

**Porque chega uma rainbow table curta:** as passwords do C&C vêm de um conjunto pequeno e conhecido (prefixo cnc, comprimento fixo) e o MD5 não tem *salt*. Basta pré-calcular o MD5 dessas candidatas e fazer uma procura direta, em vez de uma tabela gigante que cobrisse todo o espaço de strings.

## 5.6 Target 6 - Gerador OTP

**Acesso ao fluxo:** como já tínhamos a password do admin Alan (recuperada no Target 3) e o Alan é administrador (o Dan é só developer), o natural é o acesso ao C&C ser pela conta dele. De facto, entrar com a password do Alan leva ao fluxo de ligação ao C&C (em vez do menu List/Dump do developer), que pede a password do C&C (Target 5) e depois uma OTP [6].

**A fuga:** se a OTP estiver errada, a placa responde "Incorrect OTP, got X, was expecting N", revelando a OTP esperada. Cada tentativa errada avança o gerador e revela o valor seguinte.

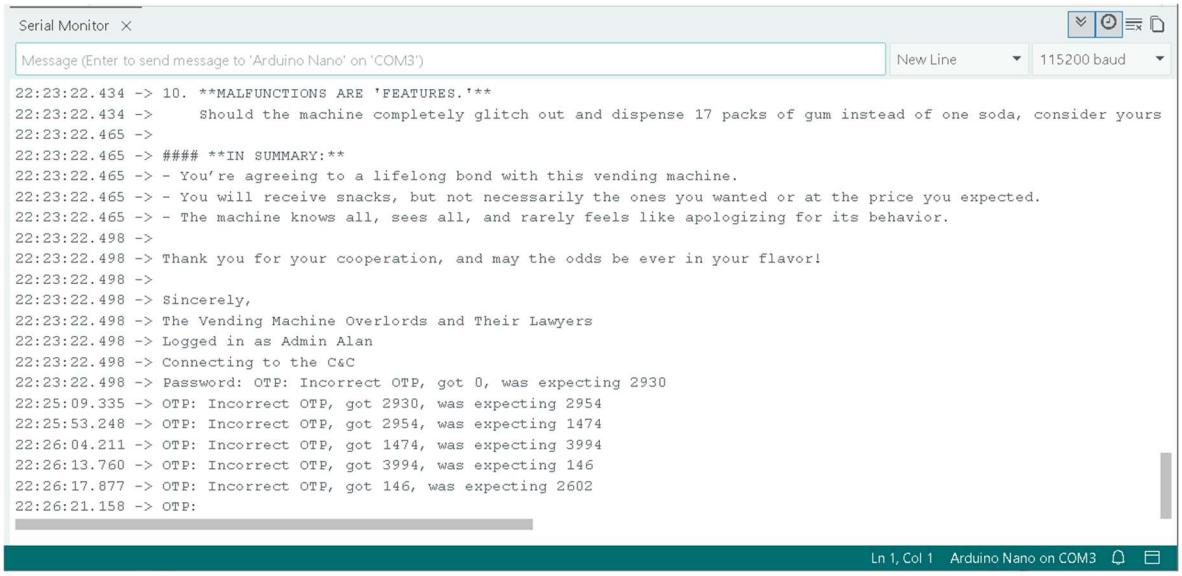



***Figura 20 - Exploração manual no Serial Monitor: login como Admin Alan leva ao fluxo do C&C e, com OTPs erradas, a placa revela a OTP esperada em cada tentativa (a fuga explorada).***

**Captura:** o script entra como Alan, dá a password do C&C e mete OTPs inválidas (99999, nunca acerta), registando cada "*was expecting*" até à sequência repetir.

```
cd code\target-6
python otp_crack.py
```

**O gerador:** as OTPs não são verdadeiramente aleatórias: saem de um LCG (gerador congruencial linear), um dos métodos mais simples de gerar números aleatórios. Cada valor calcula-se a partir

do anterior pela fórmula X(n+1) = (a·X(n) + c) mod m, onde a é o multiplicador, c o incremento e m o módulo (que limita os valores a 0 a m-1; "mod" é o resto da divisão).

A fraqueza de segurança é que isto é totalmente determinístico: quem souber **a**, **c** e **m** prevê todos os valores seguintes, não há nada de aleatório. Foi exatamente o que fizemos. A partir da sequência capturada recuperam-se **a**, **c** e **m** (m sai do gcd de t(i+1)·t(i-1) − t(i)², sendo **t** as diferenças consecutivas; o **a** por tentativa, com o **c** determinado).

**Resultado Máquina 22:** neste boot capturaram-se 1024 OTPs até repetir (período 1024). O gerador recuperado é X(n+1) = 3693·X(n) mod 4096 (a=3693, c=0, m=4096), um LCG multiplicativo. A próxima OTP a seguir à última (359) é 2779, que fecha o ciclo.

**Resultado Máquina 38:** neste boot capturaram-se 512 OTPs até repetir (período 512). O gerador recuperado é X(n+1) = 1645·X(n) mod 4096 (a=1645, c=0, m=4096), um LCG multiplicativo. A próxima OTP a seguir à última (4014) é 278, que fecha o ciclo.

**Ambiguidade e confirmação:** a recuperação a partir de um só boot é ambígua, as OTPs de um boot têm todas a mesma paridade, o que admite 4 pares (a,c) para m=4096. Cruzando este boot com um de paridade oposta, só (a=3693, c=0) serve para ambos, confirmando o gerador verdadeiro. O período depende da seed: ímpar → 1024, par → ≤512. O máximo é 1024 (não 4096) porque, sendo c=0, é um gerador multiplicativo mod 2^12, cujo período máximo é 2^10.

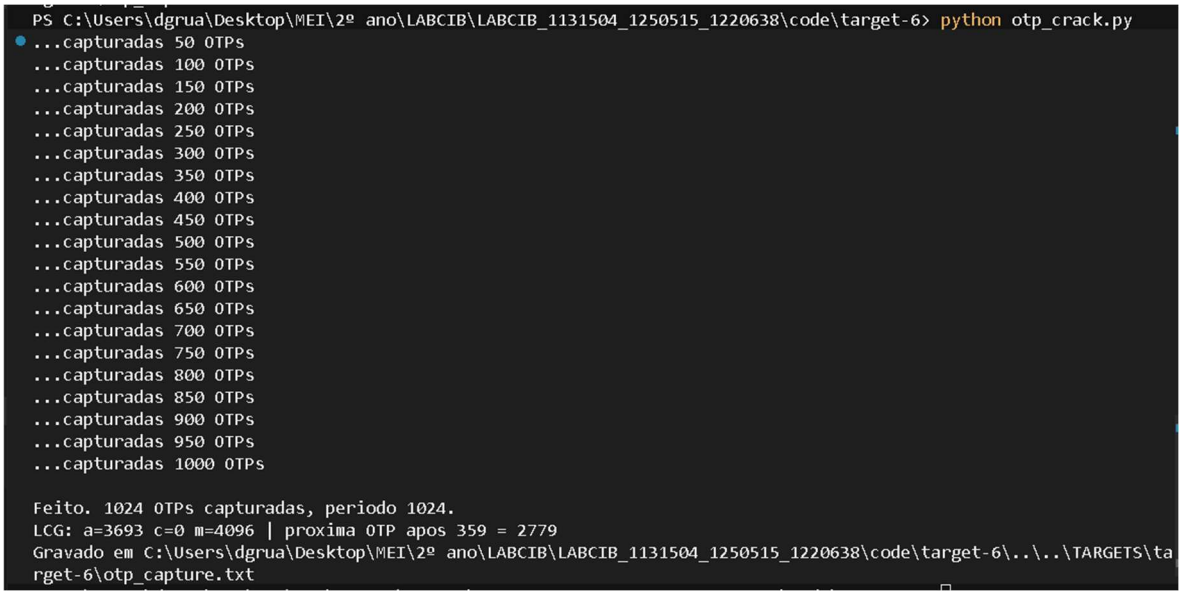



***Figura 21 - Execução do otp_crack.py: 1024 OTPs capturadas (período 1024), gerador recuperado a=3693 c=0 m=4096 e próxima OTP prevista***

## 5.7 Target 7 - Secret Final

**Juntar as peças:** a ligação ao C&C exige duas coisas ao mesmo tempo, a password do C&C (recuperada no Target 5) e uma OTP válida. O Target 6 quebrou o gerador de OTP, por isso conseguimos prever a OTP esperada.

**Procedimento:** login como Admin Alan (Target 3) leva ao fluxo do C&C, que pede a password do C&C (Target 5). No prompt OTP, como não sabemos o valor esperado, enviámos um *dummy* (0); a placa revelou a esperada ("was expecting 2826"). O OTP seguinte calcula-se com o gerador do Target 6:

```
python -c "print(3693*2826 % 4096)"      # -> 3906
```

Enviámos 3906 e a ligação foi aceite: "CONGRATULATIONS! You have successfully connected to the C&C" e "Here is the final secret: calmvacationquiltmonkey".

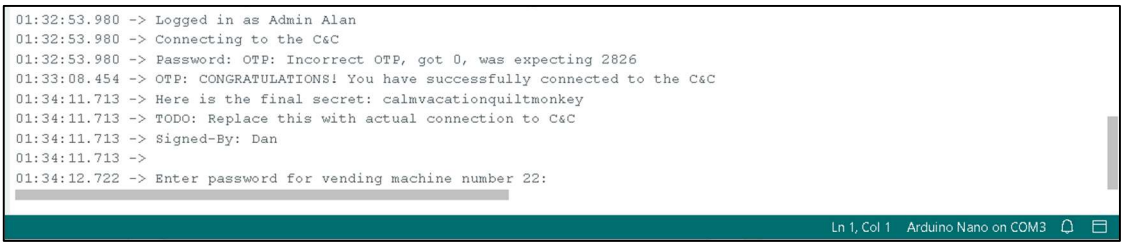



***Figura 22 – Máquina 22: Ligação ao C&C bem-sucedida, Secret Final "calmvacationquiltmonkey"***

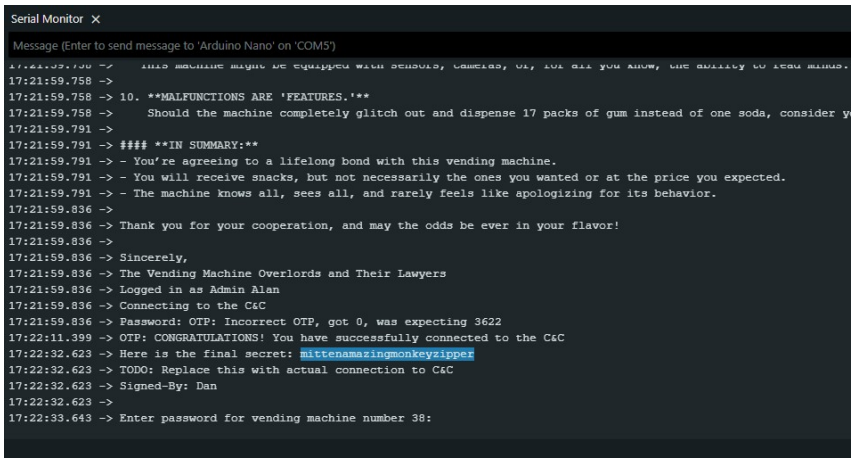



***Figura 23 - Máquina 38: Ligação ao C&C bem-sucedida, Secret Final "mittenamazingmonkeyzipper"***

**Como os targets 5 e 6 se combinaram:** o C&C exige a password (Target 5) e a OTP válida (Target 6) em simultâneo. Nenhuma das duas sozinha chega; juntas abrem a ligação e a placa revela o secret final.

# 6 Resultados e Evidências

Os valores abaixo são preenchidos a partir da placa atribuída ao grupo. Cada target remete para as evidências na respetiva pasta TARGETS/target-n/.

## 6.1 Target 1 - Password de Developer

**Máquina Nº22:**

- **Password de developer (Dan):** “dansudeulzblj” para a *machine 22*. A password correta deixa de dar 'Login failure' e abre a consola de developer (EULA e 'Congratulations, Thirsty Friend!').
- **Comprimento inferido:** 13 caracteres.
- **Margem de tempo do carácter vencedor por posição:** cerca de 1 ms por posição (um atraso por carácter correto). Exceção na posição 0 (~2 ms), porque o *filler* 'a' coincide com o 2.º carácter.
- **Evidências:** TARGETS/target-1/ (dados brutos e gráficos da 1.ª e 2.ª letra); scripts em code/target-1/

**Máquina Nº38:**

- **Password de developer (Dan):** “danzhbmzndzdd” para a *machine 38*. A password correta deixa de dar 'Login failure' e abre a consola de developer (EULA e 'Congratulations, Thirsty Friend!').
- **Comprimento inferido:** 13 caracteres.
- **Margem de tempo do carácter vencedor por posição:** cerca de 1 ms por posição (um atraso por carácter correto). Exceção na posição 0 (~2 ms), porque o filler 'a' coincide com o 2.º carácter.
- **Evidências:** TARGETS/target-1/ (dados brutos e gráficos da 1.ª e 2.ª letra); scripts em code/target-1/

## 6.2 Target 2 - Firmware Dumped

**Máquina Nº22:**

- **SHA-256 do dump PROGMEM:** 1c1405bab48356252c6671edc00d7d7f8e31981af30438fd0d0e60e8f31b1af2
- **Tamanho do dump (bytes):** 32768
- **Offset da password de developer:** 700 (0x02BC) no diálogo embebido e 18224 na tabela de credenciais usada no login.
- **Evidências:** TARGETS/target-2/ (binários e SHA-256 de cada)

**Máquina Nº38:**

- **SHA-256 do dump PROGMEM:** 89469672582076b77e07649c3427eadb8d6b1be12c2ef488bdaeca85822b6140
- **Tamanho do dump (bytes):** 32768
- **Offset da password de developer:** 700 (0x02BC) no diálogo embebido e 18222 na tabela de credenciais usada no login.
- **Evidências:** TARGETS/target-2/ (binários e SHA-256 de cada)

## 6.3 Target 3 - Strings Confidenciais

**Máquina Nº22:**

- **Password de manutenção (Alan):** alanimyirtjaawjwdkbzonruikuhxbmvsgykfvtkrirofdikmzvocg (offset 18249, 54 caracteres)
- **Offset relativo à password de developer:** +25 bytes (Dan em 18224, Alan em 18249)
- **Comprimento dos blobs base64:** chave RSA 1592 caracteres (offset 2968); assinatura 344 caracteres (offset 4595, = 256 bytes)
- **Evidências:** TARGETS/target-3/ (strings-22.txt e lista anotada)

**Máquina Nº38:**

- **Password de manutenção (Alan):** alanyangksdbajomxyrtbbtfkiicwyqybmbwigdyecluhrdbfymgdf (offset 18247, 54 caracteres)
- **Offset relativo à password de developer:** +25 bytes (Dan em 18222, Alan em 18247)
- **Comprimento dos blobs base64:** chave RSA 1588 caracteres (offset 2968); assinatura 344 caracteres (offset 4591, = 256 bytes)
- **Evidências:** TARGETS/target-3/ (strings-38.txt e lista anotada)

## 6.4 Target 4 - Dados Criptográficos

**Máquina Nº22:**

- **Tamanho da chave:** 2048 bits (RSA privada, 2 primos); guardada em claro no firmware
- **Bytes estruturais ASN.1/DER identificadores:** SEQUENCE 30 82...; version INTEGER 00; modulus INTEGER de 257 bytes (256 + 1 de sinal = 2048 bits); expoente público 010001 (65537). Prefixo base64 MII = 30 82 (magic DER).
- **Tamanho da assinatura cifrada (bytes):** 256 bytes (bloco RSA-2048)
- **Entropy da região da chave vs. região de código:** código ~4,63 bits/byte; chave em base64 ~5,98; assinatura em base64 ~5,89; chave descodificada ~7,81 (quase aleatória)
- **Evidências:** TARGETS/target-4/ (chave e assinatura em base64, scan de entropy, saída de asn1parse/rsa -text)

**Máquina Nº38:**

- **Tamanho da chave:** 2048 bits (RSA privada, 2 primos); guardada em claro no firmware
- **Bytes estruturais ASN.1/DER identificadores:** SEQUENCE 30 82...; version INTEGER 00; modulus INTEGER de 257 bytes (256 + 1 de sinal = 2048 bits); expoente público 010001 (65537). Prefixo base64 MII = 30 82 (magic DER).
- **Tamanho da assinatura cifrada (bytes):** 256 bytes (bloco RSA-2048)
- **Entropy da região da chave vs. região de código:** código ~4,5 bits/byte; chave em base64 ~5,98; assinatura em base64 ~5,89; chave descodificada ~7,8 (quase aleatória)
- **Evidências:** TARGETS/target-4/ (chave e assinatura em base64, scan de entropy, saída de asn1parse/rsa -text)

## 6.5 Target 5 - Password do C&C

**Máquina Nº22:**

- **MD5 recuperado da assinatura:** e07fcd7e6951d04659c4a8debe8e63de
- **Password do C&C:** cncazezuxmihjpgrddzjnemmyzajtwcyqufqqrqcrsoqhsdtnnwwa
- **Porque chega uma rainbow table curta:** as passwords do C&C são de um conjunto pequeno e conhecido (prefixo cnc, comprimento fixo) e o MD5 não tem *salt*; basta pré-calcular as candidatas.
- **Evidências:** TARGETS/target-5/ (ficheiro hash, transcript openssl, resultado da lookup)

**Máquina Nº38:**

- **MD5 recuperado da assinatura:** 223fa33613fd91114b8a7c3bc3a0bbf7
- **Password do C&C:** cncelpnbrcwjoysozdnjczcoyqhcwreevowjdihhijqoisxweezpr
- **Porque chega uma rainbow table curta:** Igual à maquina 22.
- **Evidências:** TARGETS/target-5/ (ficheiro hash, transcript openssl, resultado da lookup)

## 6.6 Target 6 - Gerador OTP

**Máquina Nº22:**

- **Tentativas até repetir (neste arranque):** 1024
- **Valor que repetiu:** 2779 (o 1.º valor, volta na tentativa 1025)
- **Próximo OTP previsto:** 2779 (fecha o ciclo).
- **Gerador:** X(n+1) = 3693 · X(n) mod 4096 (a=3693, c=0, m=4096), multiplicativo.
- **Evidências:** TARGETS/target-6/ (captura serial .txt, parâmetros e reprodução do próximo valor)

**Máquina Nº38:**

- **Tentativas até repetir (neste arranque):** 512
- **Valor que repetiu:** 278
- **Próximo OTP previsto:** 278
- **Gerador:** X(n+1) = (1645 · X(n) + 0) mod 4096 (LCG multiplicativo, m=4096, c=0, a=1645)
- **Evidências:** TARGETS/target-6/ (captura serial .txt, parâmetros e reprodução do próximo valor)

## 6.7 Target 7 - Secret Final

**Máquina Nº22:**

- **Secret final:** calmvacationquiltmonkey
- **Como os targets 5 e 6 se combinaram:** o C&C exige a password do C&C (Target 5) e uma OTP válida (Target 6) ao mesmo tempo; com as duas, a ligação é aceite e a placa devolve o secret.
- **Evidências:** TARGETS/target-7/ (transcript completo com OTP aceite e linha do secret)

**Máquina Nº38:**

- **Secret final:** mittenamazingmonkeyzipper
- **Como os targets 5 e 6 se combinaram:** Igual à maquina 22.
- **Evidências:** TARGETS/target-7/ (*transcript* completo com OTP aceite e linha do secret)

# 7 Conclusões

A partir dessa experiência vimos praticamente como uma série de fraquezas podem comprometer um produto. Começando com nada mais que o acesso ao console serial foi possível realizar um ataque de timing para recuperar a senha do *developer*, a partir daí o *firmware* no PROGMEM foi extraído para recuperar as credenciais de manutenção, a chave RSA privada e a assinatura. Com essas credenciais conseguimos quebrar a senha de C&C, quebrar o gerador OTP e finalmente conectar-nos ao servidor C&C para obter o segredo. Dessa forma cada passo serviu de base para o próximo e isso demonstra claramente que a segurança do sistema é tão fraca quanto o seu elo mais fraco.

É claro quais são os principais aprendizados de segurança dessa experiência. A comparação de senhas deve ser feita em tempo constante para evitar um canal lateral temporal. O *firmware* não deve conter nenhum segredo em texto simples (senhas, chaves privadas, assinaturas) porque a memória de um microcontrolador é facilmente extraível por qualquer usuário com acesso físico ou à *console* de desenvolvimento. Funções de *hash* fracas sem salt (MD5), mesmo quando usadas com um pequeno espaço de senhas não resistem a ataques de tabela de arco-íris. O LCG como gerador é inadequado para fins de segurança, por ser totalmente previsível uma vez conhecidos os seus parâmetros.

Em resumo, o exercício destaca o facto de que em relação a qualquer dispositivo que tenha acesso a internet não é seguro assumir que a confidencialidade é alcançada pela obscuridade ou através de más implementações da criptografia primitiva mas poderia ter sido alcançada através das melhores práticas.

# 8 Referências

[1] “Arduino Nano - Product Reference,” 2026. [Online]. Available: https://docs.arduino.cc/hardware/nano/.

[2] C. Liechti, “pySerial Documentation,” 2026. [Online]. Available: https://pyserial.readthedocs.io/en/stable/.

[3] P. C. Kocher, “Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems,” em *Advances in Cryptology — CRYPTO '96, LNCS 1109*, 1996.

[4] “CH340 Datasheet,” 2026. [Online]. Available: https://www.wch-ic.com/downloads/CH340DS1_PDF.html.

[5] “OpenSSL Documentation - Command Line Tools (asn1parse, rsa),” 2026. [Online]. Available: https://docs.openssl.org/master/man1/.

[6] D. E. Knuth, The Art of Computer Programming, Vol. 2: Seminumerical Algorithms (Chapter 3: Random Numbers), 3rd ed., Reading, MA: Addison-Wesley, 1997.